NEXUS · DOCUMENTATION PACKAGE · DELIVERABLE 05

Backend and Data Schema

Entities, relationships, key fields, storage locations and data flows on the server and the device.

Document control

Document ID	NX-05-DATA
Version	2.0
Date	2026-11-05
Status	Draft for team review (not approved)
Intended readers	Developers changing the API or Room schema; assessor checking the data design
Evidence baseline	Branch New-Changes: commit 716bedb plus the uncommitted Part 3 working-tree changes of 2026-11-05 (v1.0 analysed 3a3f275)
Prepared by	AI-assisted (Claude Code) from repository evidence; team review pending
Approved by	Unknown No approval recorded

Evidence labels used in this document: Implemented verified in code, configuration or test output; Inferred reasoned from evidence but not executed; Recommendation proposed, not implemented; Open question needs a decision; Unknown no evidence available; Estimate planning figure, not a measurement.

Contents

1. 1 Storage overview
2. 2 Server data model
    1. 2.1 User (ENT-USER)
    2. 2.2 Project (ENT-PROJECT)
    3. 2.3 Task (ENT-TASK)
    4. 2.4 Habit (ENT-HABIT)
3. 3 Device (Room) schema, version 5
    1. 3.1 Migration history
4. 4 Data flows
5. 5 Data retention and privacy notes

1 Storage overview

Where Nexus data lives

Store	Location	Contents	Protection
API document store	`DATABASE_PATH` (default `backend/nexus-data.json`; `/data/nexus-data.json` in the container)	users, projects, tasks, habits, sync receipts	File-system permissions only; not encrypted at rest. Git-ignored.
Room database	App-private `nexus_database`, schema version 5	Cached projects, tasks, habits, dashboard counts, pending upload queue	App sandbox; excluded from Android backup since v2.0 (AUD-05 fixed)
Encrypted preferences	`nexus_secure_prefs`	JWT (`jwt_token`)	AES-256 GCM/SIV via AndroidX Security; excluded from backup
Plain preferences	Settings store; `cached_profiles`	`theme_mode`, `notifications_enabled`, `language`, language-sync flag; profile JSON per owner (`profile:<owner>`)	App sandbox; excluded from backup

2 Server data model

`server.js` keeps four in-memory `Map`s, persisted as one JSON document `{ users: [...], projects: {userId: [...]}, tasks: {userId: [...]}, habits: {userId: [...]} }`. Ownership is structural: every collection is keyed by the owner's user id, so there are no cross-user references **Implemented**.

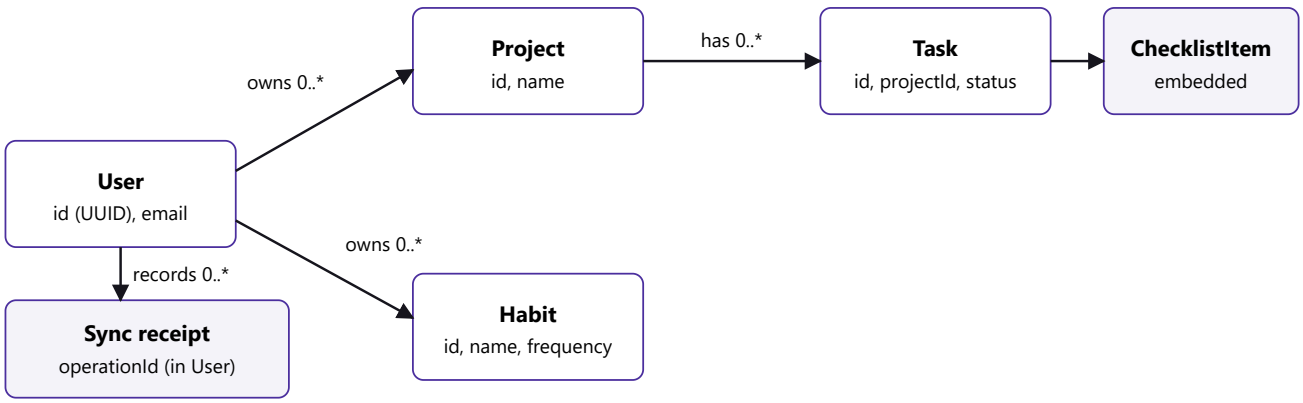


Figure 3. Logical entities. Tasks are stored per user and linked to a project by `projectId`; checklist items and labels are embedded arrays.

2.1 User (ENT-USER)

Field	Type	Rules
`id`	UUID string	`randomUUID()` at registration; JWT `sub`

Field	Type	Rules
`email`	string	Valid email, stored lower-case since v2.0; unique and matched case-insensitively **Implemented**
`displayName`	string	Non-empty
`passwordHash`	string	bcrypt cost 12. A non-bcrypt value is compared as plain text once and then rehashed (AUD-16). Never returned by the API.
`language`	string	Default `"en"`; any string accepted by the API, the app sends en/zu/tn
`notificationsEnabled`	boolean	Default `true`
`syncOperations`	string[]	Applied `operationId`s; grows without limit (AUD-07)
`googleSub` (v2.0)	string	Google account subject; set on first Google sign-in. Accounts created by Google have no `passwordHash`.
`deviceTokens` (v2.0)	string[]	FCM registration tokens, at most 10, newest last; a token is removed from any other account when registered

2.2 Project (ENT-PROJECT)

Field	Type	Rules
`id`	UUID string	Server-generated (REST) or device-generated (sync)
`name`	string	Non-empty
`description`	string | null	Optional
`createdAt`, `updatedAt`	ISO-8601 timestamp	Set by the server

Deleting a project deletes its tasks (REST and sync paths) **Implemented**.

2.3 Task (ENT-TASK)

Field	Type	Rules
`id`, `projectId`	UUID strings	The project must belong to the same user (404 on REST, 409 on sync)
`title`	string	Non-empty
`description`	string | null	Optional
`dueDate`	string | null	Pattern `yyyy-MM-dd` (format only; the calendar date is not checked)
`priority`	enum	`NONE` (default), `LOW`, `MEDIUM`, `HIGH`
`status`	enum	`TODO` (default), `IN_PROGRESS`, `DONE`
`isCompleted`	boolean	Derived: always `status === "DONE"` after a write
`labels`	string[]	Free text
`checklist`	{id?, title, isCompleted?}[]	Title non-empty

Field	Type	Rules
`createdAt`, `updatedAt`	ISO-8601	Server-set

2.4 Habit (ENT-HABIT)

Field	Type	Rules
`id`	UUID string	
`name`	string	Non-empty
`description`	string | null	Optional
`frequency`	enum	`DAILY` (default), `WEEKLY`, `CUSTOM`
`targetDays`	number[]	Accepted but not used by the app's scoring
`completedDates`	string[]	`yyyy-MM-dd` by convention; no format validation on the server
`createdAt` (+ `updatedAt` via sync)	ISO-8601	

3 Device (Room) schema, version 5

Room entities in data/Database.kt and data/SyncManager.kt

Table	Primary key	Columns
`projects`	`id`	`userId`, name, description, createdAt, updatedAt
`tasks`	`id`	`userId`, projectId, title, description, isCompleted, dueDate, priority, status, labels (JSON), checklist (JSON), createdAt, updatedAt
`habits`	`id`	`userId`, name, description, frequency, targetDays (JSON), completedDates (JSON), createdAt
`dashboard`	`userId`	projects, tasks, activity (counts)
`pending_changes`	`sequence` (autoincrement)	`userId`, operationId (UUID), resource (projects|tasks|habits), entityId, action (create|update|delete), payload (JSON)

List columns are stored as Gson JSON strings by `TaskConverters`. `userId` is the cache owner id: the JWT `sub`, or a hash of the token if no `sub` exists. No foreign keys or indices are declared **Implemented**.

3.1 Migration history

Migration	Change	Data effect
1→2, 2→3	Not present in code (version 2 added task fields, per commit 8b31aa3)	Upgrading from v1 or v2 would fail to open the database (AUD-12)
3→4	Adds `userId` to projects, tasks and habits; rebuilds `dashboard` keyed by user	Deletes all cached rows (they could not be attributed to an owner)
4→5	Creates `pending_changes`	Keeps existing rows

4 Data flows

Principal data flows

ID	Flow	Path
DF-01	Local write	ViewModel → Repository → `SyncManager.save()` → Room entity + `pending_changes` (one transaction). See SEQ-01.
DF-02	Upload	`pending_changes` (ordered) → `POST /api/sync` → in-memory map → `saveDatabase()` (`.tmp` then `rename`) → receipt stored → queue row deleted
DF-03	Refresh	`GET /projects` + `GET /projects/:id/tasks` for each project (N+1 requests) → replaces the account's cached projects and tasks → dashboard counts computed on the device
DF-04	Account deletion	`DELETE /users/me` → server removes the user and all collections → app purges that owner's Room rows and queue under the sync mutex
DF-06	Real-time push (v2.0)	Successful `/api/sync` → `notifyUser()` → FCM multicast to the user's other device tokens → `NexusMessagingService` shows a notification and refreshes the Room cache
DF-07	Google SSO (v2.0)	Credential Manager → Google ID token → `/api/auth/google` → Google key verification → user found/created/linked → Nexus JWT
DF-05	Export	Habits (cache or API) → CSV string in memory → Android share sheet. Nothing is written to shared storage.

5 Data retention and privacy notes

- Personal data held: email, display name, password hash, the user's own content and, since v2.0, the Google account subject and FCM device tokens. Google (sign-in) and Firebase (push delivery) act as service providers; no analytics or ads **Implemented**.
- Retention: data is kept until the user deletes it or the account. There are no server-side backups or expiry **Implemented**.
- Logs: Pino redacts password fields, `passwordHash` , `token` and the authorization header. pino-http's default serializers do not log request bodies, so emails in bodies do not reach the logs, but remote IP addresses are logged **Implemented**.